

MATERIAL EXTRA

Trabalhando com
Módulos



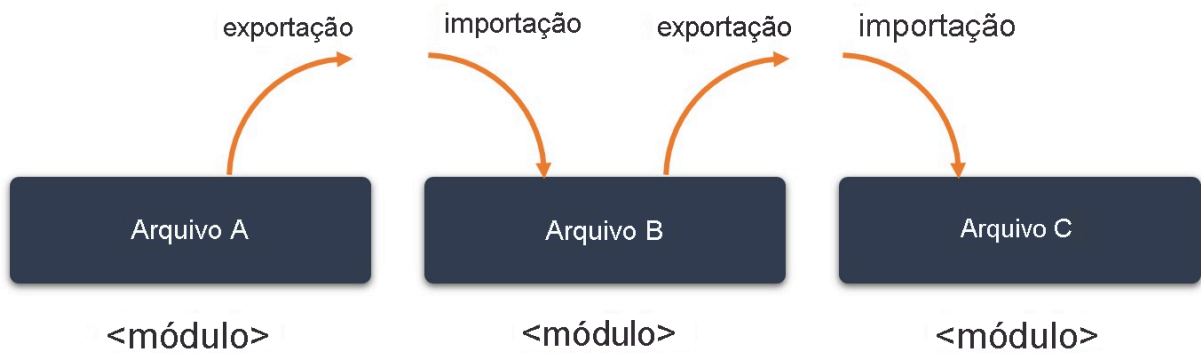
ORGANIZAR CÓDIGO COM MÓDULOS

Os módulos fornecem uma forma de organizar e categorizar seu código, permitindo que você agrupe o código relacionado. Além disso, quando o código está dentro de um módulo, ele é extraído do escopo global e para o escopo do módulo. Isso pode ajudá-lo a evitar conflitos de nomenclatura entre componentes.

 **DICA:** Os módulos têm sido um recurso do JavaScript desde ES6, portanto, eles também têm suporte pelo TypeScript. Antes de ES6, as versões anteriores do TypeScript faziam referência a módulos como "módulos externos".

Você pode exportar qualquer declaração (como uma variável, uma função, uma classe, um alias de tipo ou uma interface) adicionando a palavra-chave **export** ou importá-la usando a palavra-chave **import**. Qualquer arquivo contendo uma instrução `import` ou `export` de nível superior é considerado um módulo.

A instrução `export` torna explicitamente um componente em um módulo disponível para outros módulos, enquanto a instrução `import` permite que você consuma esse componente de outro módulo. Os módulos são declarativos, as instruções `import` e `export` no nível de arquivo descrevem as relações entre elas.



Por que usar?

- Facilita a **reutilização de código** entre arquivos.
- Promove uma melhor **organização e manutenção** do código.
- Evita a repetição de código, já que módulos podem ser reutilizados.
- Ajuda a dividir o código em **módulos menores**, mais fáceis de entender.

TIPOS DE EXPORTAÇÃO

O TypeScript oferece suporte a dois tipos de exportação: **nomeada** e **default**.

Exportação nomeada

Permite exportar várias variáveis, funções ou classes de um único arquivo. Na importação, você precisa usar o nome exato do item exportado.

mathUtils.ts

```
export const add = (x: number, y: number): number => x + y;  
export const subtract = (x: number, y: number): number => x - y;
```

app.ts

```
import { add, subtract } from './mathUtils';  
  
console.log(add(5, 3)); // Saída: 8  
console.log(subtract(10, 4)); // Saída: 6
```

NOTA: Se você quiser, pode renomear as importações usando a palavra-chave **as**.

app.ts

```
import { add as sum } from './mathUtils';  
console.log(sum(2, 3)); // Saída: 5
```

Combinando Exportação Nomeada e Padrão

Você pode combinar exportação padrão com exportações nomeadas no mesmo arquivo.

É importante lembrar que você só pode ter uma exportação padrão por arquivo e que não existem números máximos de exportações nomeadas por arquivo, mas é sempre recomendável manter uma boa separação e organização dos códigos do projeto.

math.ts

```
export const multiply = (x: number, y: number): number ⇒ x * y;

export default function divide(x: number, y: number): number {
  return x / y;
}
```

app.ts

```
import divide, { multiply } from './math';

console.log(multiply(2, 3)); // Saída: 6
console.log(divide(10, 2)); // Saída: 5
```

Importação de Tudo (Import All)

Você pode importar todo o conteúdo de um módulo usando `*` `as`.

mathUtils.ts

```
export const add = (x: number, y: number): number => x + y;

export const subtract = (x: number, y: number): number => x - y;
```

app.ts

```
import * as math from './mathUtils';

console.log(math.add(5, 3));      // Saída: 8
console.log(math.subtract(10, 4)); // Saída: 6
```

RESUMO

- **export:** Torna funções, classes, variáveis, interfaces ou tipos disponíveis para outros arquivos.
- **import:** Permite usar elementos exportados de outros arquivos.
- **Exportação Nomeada:** Múltiplas exportações por arquivo.
- **Exportação Padrão:** Apenas uma exportação por arquivo, com nome livre na importação.
- **Importação de Tudo:** Importa todo o conteúdo de um módulo como um objeto.

BONS ESTUDOS